

LetterLab

Ethan Balda and Marcus Moore

Ethan:  LetterLabFinalPart1.webm

Marcus:

 Final Project – GameActivityExp...

https://github.com/EthanBalda/FinalProject_LetterLab.git

Names and GitHubs

Marcus Moore

GitHub: marcusm77

Ethan Balda

GitHub: EthanBalda

Project Proposal

Concept: Create a puzzle game based around sorting a set of letters into alphabetical or reverse-alphabetical order. Each letter has a unique rule that must be used to sort the whole set. The game is based on an old gamejam game made by Marcus. This will effectively port the game to mobile devices, while also fully fleshing out the game's mechanics.

Display: The app will have a starting screen that will display multiple buttons such as levels, help, a dictionary, and settings.

The dictionary displays the uncovered rules for each letter.

In a level, the player views the unsorted grids of letters, the level's name, a retry, cancel and home button, and a move counter. The retry button resets the current puzzle. The cancel button deselects the current letter. Tapping anywhere on the screen not containing a letter or button also accomplishes this. The home button brings the user back to the main menu.

Upon completing a puzzle, a small menu appears displaying your moves, the moves to get par, and the moves to get a medal. The player can then continue to the next level, retry the current level, or return to the game's main menu.

Gameplay

The player selects a selectable letter, then another letter to interact with. These interactions count as moves, which are counted in the corner. The goal of the player is to turn a garbled set of letters into alphabetical order in as few moves as possible. Note that some letters have the same effect regardless of the letter selected, such as 'm'. These still ask a player to select another letter as a means of confirming the move.

Completing a level under a specified par unlocks the next. Doing particularly well on a puzzle also rewards a medal.

The first sets of levels will unlock linearly. A set of challenge levels will be rewarded after the main game's completion depending on how many medals the player has accrued.

The Letters:

a; Apathetic

Does nothing. Other letters can interact with it.

b; Basic : ab a -> aab OR ba a

Swaps places with a cardinally adjacent letter.

c; Complex : aacaa -> caaaa OR aaaac

Swaps places with a letter cardinally adjacent letter one slot away from it. One more space than b.

d; Delete : adb -> ad

Deletes whatever letter tries to interact with it. Has some weird interactions

e; Eat : abcec -> ab cc

Eats any adjacent letter, deleting that letter and becoming it.

f; Force : abfcc -> bfcca OR cabfc

Moves the entire string or grid cardinally one space. If a letter would be knocked off the string, it wraps back to the start.

g; Grab : abcagbb -> bcaagbb OR abacagbb OR acabagbb

Grabs a cardinally adjacent letter, pulls it adjacent to 'g', then moves all other letters back to fill space.

h; Hold (This may need to be removed with grids) : a**h**b -> a**h**^b_{__}

Holds a selected letter over its head, effectively removing it from the string or grid. Can swap out whatever is held with another letter. If it grabs a 'd', the 'h' and whatever it was holding get deleted, and the 'd' moves to the 'h' 's location. Can only hold and swap any adjacent letter.

i; Immovable : a**c**bi -> a**c**i

Cannot be interacted with by anything other than 'n'.

j; Jump : abc**j**cba -> jbc**a**cba OR abc**a**c**j**

Like 'c', but with 1 more range.

k; Kick : b**k**abba -> b**k**bb**a**

The exact opposite of 'g'. Kicks any cardinally adjacent letter to the edge of the grid or string, then shifts every letter towards it to fill in the gap.

l; Lovely (These should probably be ditched if we commit to grids.) : bacb**l**ccbb -> **b**bac**l**cb**b**c

Pulls every cardinal letter into it whenever it is selected. Letters that would move to the selected 'l's slot instead go to the edges.

m; Monstrous (These should probably be ditched if we commit to grids.) : bacb**m**ccbb -> acbbmb**c**cb

Opposite of 'l', they scare every cardinal letter one slot away. Any letters that would go off the grid, instead end up in the empty adjacent slots.

n; No : aabncn -> aacnb BUT NOT aabcn -> acnba

Adjacent letters to an 'n' effectively become 'a's. They have no special abilities anymore. This also applies to 'i's and makes this letter the only way to interact with 'i's. If two or more 'n's are next to each other, they 'no' each other effectively making both work as 'a's until they are no longer adjacent. There is no negative stacking, so 3 'n's cancel each other just like 2 'n's.

o; Onomatopoeia : abaobba -> obaabba OR aboabba OR abaabbo

Can swap places with any selected vowels, except 'i'.

p; Purge : aipo -> a___

Deletes self and cardinal neighbors when selected. Only deletes letters when it is selected first. It can be safely swapped with or interacted with by other letters.

q; Queen : (I hope this is self explanatory because listing all the possible moves in even a small puzzle would make this letter look way more complicated than it is.)

Can swap with any letter, barring 'i's and 'd's

r; Remove : accdrca -> a__da_a

Removes every instance of a selected letter in a level, then transforms itself into an 'a'. Can only remove a letter that is adjacent to it. Can remove 'i's without issue.

s; Spin : bsa -> asb

Swaps any of the letters adjacent to it with the letter in the opposite adjacent slot. Note that while both adjacent letters are

selectable, they have the same result.

t; Toggle : btc -> atc OR atb

Transforms the selected letter into the previous letter alphabetically. 'a's become 'z's. Can only 'toggle' letters adjacent to it.

u; Undo : buc -> cuc OR bud

The exact opposite of 't'. Changes an adjacent letter to its next letter alphabetically. 'z's become 'a's.

v; Virus : a**v**uac -> a**v**uac OR a**v**uac

Exactly like a 'b', but whenever it swaps with a letter, that other letter becomes a 'v'. 'i's are of course immune to this. 'd's delete this letter then become a 'v'. 'h' that hold a 'v', delete that 'v' then become a 'v' themselves. t and u transform the letter, then become 'v's themselves.

w; Waltz : a**w**iua -> a**w**iua -> a**w**iua

Always swaps diagonally up left when selected. Jumps 'i's but does not swap with them. Can only select the diagonal upper left letter. When at the edge, it can select the letter at the other edge to loop around.

x; Xerox : caxb -> baxb

Selects two letters. Transforms the first letter into the second letter. Can transform letters into 'i's. Cannot transform 'i's. Same goes for 'n's. If it tries to transform a 'v', it instead makes both letters 'v's.

y; Yeet : baaycib -> byaacib OR baacyib

Jumps the number of spaces equal to the number of vowels in the cardinal direction it wants to go.

z; Zap : (Like the Queen, showing an example makes this look more complicated than it actually is.)

Can swap any two letters with each other.

Further Clarifications on Select Letters: D

e= both get deleted.

f = deletes the letter that would move into it.

g= d gets pulled to it like a normal letter, then the g is deleted.

h= deletes the currently held letter and the h, but moves to the h's spot.

k= gets thrown to the edge like other letters. Deletes the letter at the edge and not the 'k'

l = instead of going back to the edge, it deletes the 'l' if it goes all the way to 'l'.

m=just like 'l' except it gets stuck at the edge and deletes incoming letters.

n= no longer deletes anything.

p= deletes 'd' without issue.

r= removes all 'd's from the level, but is deleted in the process rather than becoming an 'a'.

s= deletes the letter it would swap with, not the 's'.

v= deletes the 'v' but becomes a 'v'.

x= deletes whatever the other letter is. Still transforms into the other letter if it was selected first.

z= Moves to the other letter's location but deletes the other letter.

Further Clarifications on Select Letters: H

- d= deletes the 'h' and whatever letter it was holding. The 'd' moves to the 'h's position.
- h= if this 'h' you are trying to hold also has a letter, that letter stays held by the 'h' held by the 'h'. The consequences for this will be something we regret later.
- n= if picked up, the 'h' can no longer pick up any other letters.
- r = if the selected letter is held by 'h', the letter is removed allowing 'h' to hold a new letter.
- v= deletes this letter, but the 'h' and the held letter both become 'v's.
- x= holding this does nothing special. However, if an 'h' holding a letter is xerox'd, the held letter is also copied over to the new 'h'.

Further Clarifications on Select Letters: V

a/b/c/j/q/s/z= swap positions like normal, then the other letter becomes a 'v'.

d= deletes the 'v' then becomes a 'v'.

e= becomes a 'v' without deleting the original 'v'.

g= it gets pulled to the 'g' like a normal letter, then transforms the 'g' into a 'v'.

h= swaps places with the held letter like normal. The held letter then becomes a 'v'. The 'h' then also becomes a 'v', deleting the held 'v'.

k= kicks the 'v' to the edge. The letter at the edge becomes a 'v' then every letter moves to fill the gap.

l = Instead of going to the edge when reaching the 'l', it transforms the 'l', then all letters adjacent to the 'l'. Note that this means the rest of the string or grid do not move closer to the 'l' when a 'v' would move into the 'l'.

m= when it wraps around an edge back to 'm', the 'm' becomes a 'v' and all letters adjacent to the 'm' also become a 'v'.

n= if next to the 'v', the 'v' cannot infect any letters. If the 'v' is swapped away from the 'n', it still cannot infect anything until its next turn. If a letter is adjacent to 'n' at the end of its swap with a 'v'. It does not get infected.

w= if a 'w' swaps with it, the 'w' does become a 'v'.

x = both letters become 'v's regardless of what the other letter is.

z = treat the swap as any other 'v' swap.

